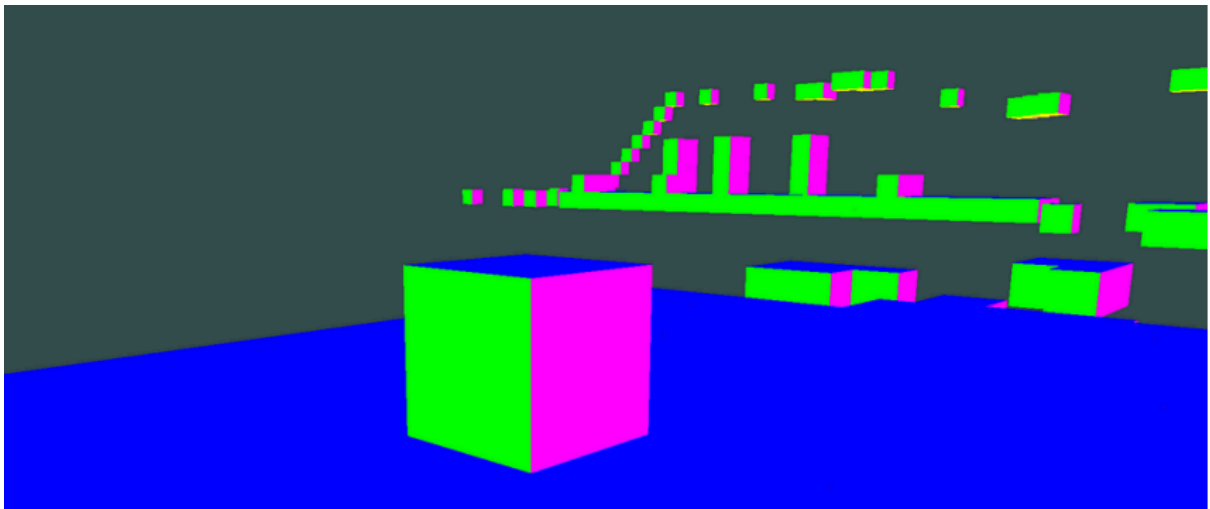

Projet TSI – Synthèse d'Images

Auteurs : Lucic Arthus, De la chapelle Jean



Sommaire

- I. Objectif
 - II. Notre projet
 - 1. Idée du jeu
 - 2. Environnement
 - 3. Les touches
 - III. Détails du projet
 - 1. Personnage principal
 - 2. Mise en place de la caméra
 - 3. Construction de la carte
 - 4. Déplacements et rotation
 - 5. Gravité
 - 6. Collisions
 - 7. Shaders
 - 8. Mode création
 - 9. Retour au point de départ
 - IV. Conclusion
-

I. Objectif

Ce projet a été réalisé dans le cadre du module de TSI – Synthèse d'images.

L'objectif était de créer un jeu 3D simple en OpenGL comportant plusieurs éléments techniques :

- Objets 3D
 - Transformations
 - Gestion d'une caméra libre
 - Simulation de la physique (gravité, collisions)
 - Réutilisation d'objets
 - Création d'un jeu minimaliste avec interactions utilisateur
-

II. Notre projet

1. Idée du jeu

Nous avons réalisé un jeu en 3D dans lequel un personnage (représenté par une sphère) évolue sur une carte composée de cubes.

Le joueur peut se déplacer, tourner, sauter, et subir la gravité.

Objectif : réaliser le parcours entre les deux plateformes dans le temps le plus court.

L'inspiration vient des maps de jump de Minecraft.

2. Environnement

Le jeu est codé en Python avec OpenGL, en utilisant :

- La bibliothèque **Pyrr** pour la gestion des matrices et de la caméra
- La bibliothèque **ModernGL** pour les shaders

- Des shaders **GLSL personnalisés** (`shader.vert` et `shader.frag`)
- Des fichiers **JSON** pour gérer la carte

3. Les Touches

NB: le chrono se lance/coupe automatiquement en passant sur le rebord de la plateforme de start/fin.

MODE JEU (mode de départ)

Déplacement joueur : zqsd

Saut : espace

MODE EDITEUR (appuyer sur M pour activer)

Ajouter bloc : E

Supprimer bloc : R

Afficher la map enregistré : L

Enregistrer la map actuelle dans un json : P

MODE DEPLACEMENT LIBRE (appuyer sur N pour activer)

Déplacement : zqsd

Déplacement haut/bas : A,W

III. Détails du projet

1. Personnage principal

Le personnage principal du jeu est représenté par une sphère 3D, générée dynamiquement à l'aide d'une fonction utilitaire qui crée les sommets et indices OpenGL d'une sphère. Cette sphère est contrôlée par le joueur à l'aide du clavier (déplacements, saut, etc.) et interagit avec l'environnement (plateformes, gravité, collisions). Le choix d'une sphère permet d'avoir un rendu simple, lisible et facilement manipulable dans l'espace 3D, tout en gardant la logique de collision et de mouvement intuitive pour un jeu de plateforme/parkour. Toutes les transformations (position, rotation) appliquées à la sphère reflètent directement les actions du joueur dans le jeu.

2. Mise en place de la caméra

Une caméra libre est attachée au joueur.

Elle suit le personnage à une certaine distance et regarde toujours vers lui, ce qui permet une vue fluide à la 3e personne.

La matrice de vue est générée avec `pyrr.matrix44.create_look_at`.

3. Construction de la carte

On a d'abord affiché nos premiers éléments en python puis au vu du parcour à réaliser nous avons opté pour un éditeur. Nous avons ajouté un mode edit permettant de rajouter des blocs, de les supprimer et d'enregistrer les ajouts dans un json.

Le monde est construit à partir de fichiers JSON (`map.json`) qui contiennent les blocs à afficher.

Chaque bloc est un cube 3D avec coordonnées, couleur et hauteur, instancié plusieurs fois à l'aide d'un VAO.

Une fois la map terminée nous l'avons copié les données obtenues dans un nouveau json pour faire la map définitive.

A noter que nous avons laissé le mode edit dans la version finale.

4. Déplacements et rotation

Le joueur peut se déplacer dans toutes les directions, en fonction de sa rotation :

Le pad/souris contrôle l'orientation, zqsd contrôle les déplacements.

Les translations (déplacements) du personnage principal sont gérées principalement par la fonction ``compute_movement``. Cette fonction lit les touches pressées (WASD pour avancer/reculer/déplacer latéralement, et d'autres pour le mode libre) et calcule un vecteur de déplacement selon l'orientation de la caméra (``yaw``). En mode normal, le déplacement est projeté sur le sol et passe par la fonction ``check_collisions`` pour éviter de traverser les plateformes ou obstacles : la position du personnage (``self.cube_pos``) est alors mise à jour uniquement si le mouvement est possible.

Afin de mieux créer la map nous avons ajouté un mode de déplacement libre sans gravité:

En mode déplacement libre, le personnage se déplace directement dans l'espace 3D sans collision. La gravité, appliquée dans ``apply_gravity`` ou dans la boucle principale, modifie la composante verticale du déplacement (axe Y).

Ainsi, les translations du personnage résultent d'une combinaison du mouvement horizontal (touches directionnelles), du saut (modification de la vitesse verticale), et de la gestion des collisions pour garantir un déplacement réaliste et cohérent dans l'environnement du jeu.

5. Gravité

La gravité dans ce jeu est simulée grâce à la variable ``self.gravity`` (valeur négative, par exemple -0.04) et la méthode ``apply_gravity``. À chaque frame, la gravité est ajoutée à la vitesse verticale du personnage (``self.cube_velocity_y``), ce qui fait diminuer progressivement sa position sur l'axe Y (``self.cube_pos[1]``). Cela permet de reproduire l'effet d'une chute : plus le personnage reste en l'air, plus il accélère vers le bas.

Quand le personnage touche le sol ou une plateforme (détecté via les collisions), la vitesse verticale est remise à zéro pour l'empêcher de traverser les blocs. Cette gestion simple mais efficace donne une sensation réaliste de saut et de chute dans le jeu.

6. Collisions

Les collisions dans ce jeu sont gérées par la fonction ``check_collisions``.

À chaque déplacement, la position future du personnage est testée séparément sur chaque axe (X, Z, puis Y) pour voir si elle entre en contact avec un bloc du décor (plateforme).

Pour chaque axe, la fonction vérifie si le volume du personnage (un cube autour de la sphère) chevauche celui d'une plateforme : si c'est le cas, le déplacement sur cet axe est annulé (le personnage reste à sa position précédente sur cet axe).

Pour l'axe Y, si le personnage tombe sur une plateforme, il est replacé juste au-dessus ; s'il monte et touche un plafond, il est bloqué. Enfin, la fonction empêche aussi le personnage de passer sous le sol.

Ce système évite que le joueur traverse les blocs et permet des mouvements réalistes sur les plateformes.

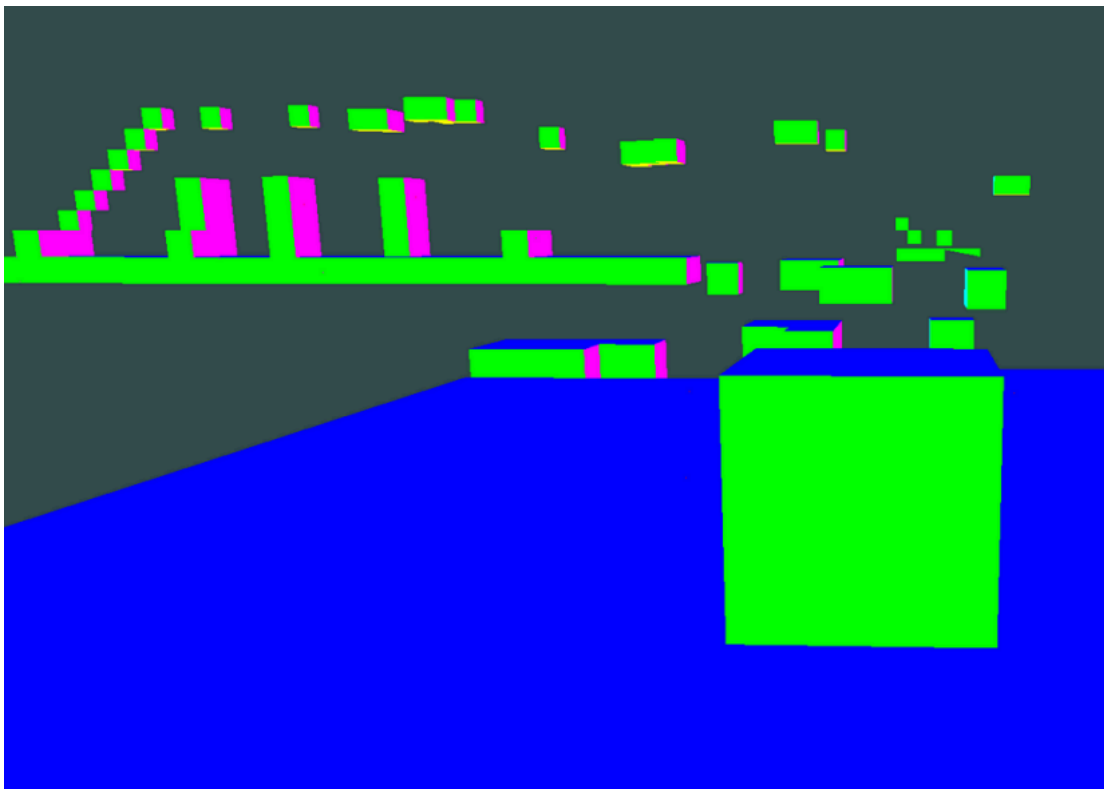
7. Shaders

Deux shaders GLSL sont utilisés :

- `shader.vert` : gère les transformations et les normales
- `shader.frag` : applique une couleur et un éclairage simple

Chaque sommet contient ses coordonnées, normales, couleur et coordonnées UV. Les objets sont dessinés via un **VAO** et un **VBO** avec données interlacées.

8. Mode Création (Éditeur de Carte)



Le projet intègre un **éditeur de carte** (`map_editor.py`) qui permet :

- D'ajouter ou supprimer des blocs sur une grille 3D
- De sauvegarder la carte en JSON
- De recharger la carte dans le jeu

Ce mode création repose sur la manipulation directe du modèle de carte du moteur. L'interface permet également de prévisualiser les modifications avant enregistrement, ce qui rend l'environnement entièrement personnalisable.

9. Retour au point de départ en cas de chute

Un système de **respawn automatique** évite que le personnage tombe indéfiniment. Si la position passe sous un seuil (ex : $z < -5$), le joueur est replacé au point de départ.

Cela améliore :

- L'expérience utilisateur
- Le réalisme (effet de chute contrôlée)
- Le gameplay (punition légère au lieu d'un échec total)

IV. Conclusion

Ce projet nous a permis de :

- Comprendre le pipeline graphique OpenGL moderne
- Manipuler des objets 3D avec shaders
- Implémenter des mécaniques de physique simples (gravité, collisions)
- Créer un moteur de jeu 3D évolutif en Python

Améliorations possibles

Nous aurions souhaité ajouter l'affichage des résultats du chrono sur l'écran mais nous aurions eu besoin d'installer d'autres bibliothèques, nous avons donc abandonné cette idée pour se limiter à l'affichage dans la console.